

Analiza stabilności numerycznej algorytmów na przykładzie obliczania wartości funkcji pierwiastkowych

Metody Obliczeniowe w Nauce i Technice

Piotr Sączawa

11 marca 2026

Cel doświadczenia

- Analiza porównawcza dwóch matematycznie tożsamyh sformułowań funkcji:

$$f(x) = \sqrt{x^2 + 1} - 1, \quad g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

- Wyznaczenie wpływu skończonej precyzji arytmetyki zmiennoprzecinkowej (IEEE 754) na wynik obliczeń.
- Porównanie zachowania typów float, double oraz long double.
- Badanie stabilności dla argumentów skrajnych ($x \rightarrow 0$ oraz $x \rightarrow \infty$).

Specyfikacja techniczna

- **System operacyjny:** Windows 11.
- **Procesor:** Architektura x86_64 (obsługa jednostek FPU i instrukcji SSE/AVX).
- **Kompilator C++:** GCC (MinGW-w64) – standard C++17.
- **Środowisko obliczeniowe Python:** Python 3.15 (biblioteki: pandas, matplotlib, numpy).
- **Referencja wysokiej precyzji:** Biblioteka decimal (ustawiona precyzja: 100 cyfr znaczących).

Uwaga dot. typu long double

W użytej architekturze (x86_64) i kompilatorze GCC, typ long double zajmuje zazwyczaj 12 lub 16 bajtów w pamięci, oferując 80-bitową precyzję (rozszerzony format x87), co pozwala na dokładniejszą analizę błędów typu double.

Problem I: Katastrofalne obcięcie ($x \rightarrow 0$)

Dla bardzo małych wartości x , wyrażenie $\sqrt{x^2 + 1}$ dąży do 1.

Mechanizm błędu w $f(x)$

Gdy $x^2 < \epsilon_{machine}$, wartość pod pierwiastkiem jest reprezentowana jako dokładnie 1.0 ze względu na ograniczoną liczbę bitów mantysy.

- Następuje odejmowanie dwóch bliskich sobie liczb ($1.0 - 1.0$).
- Najbardziej znaczące bity wyniku zostają wyzerowane.
- Wynik traci wszystkie cyfry znaczące (wynik 0 dla $x \neq 0$).

Rozwiązanie: Przekształcenie do postaci $g(x)$ eliminuje odejmowanie w liczniku na rzecz dodawania w mianowniku, co jest operacją stabilną numerycznie.

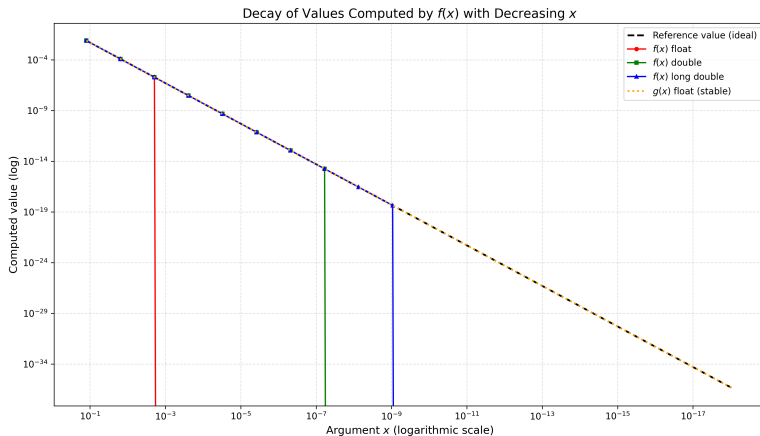
Problem II: Przepiętnienie zakresu ($x \rightarrow \infty$)

Typy danych IEEE 754 mają ściśle określony zakres wykładnika (cechy).

Typ	Max wykładnik	Max wartość (x^2)
float (32-bit)	2^{127}	$\approx 3.4 \cdot 10^{38}$
double (64-bit)	2^{1023}	$\approx 1.8 \cdot 10^{308}$

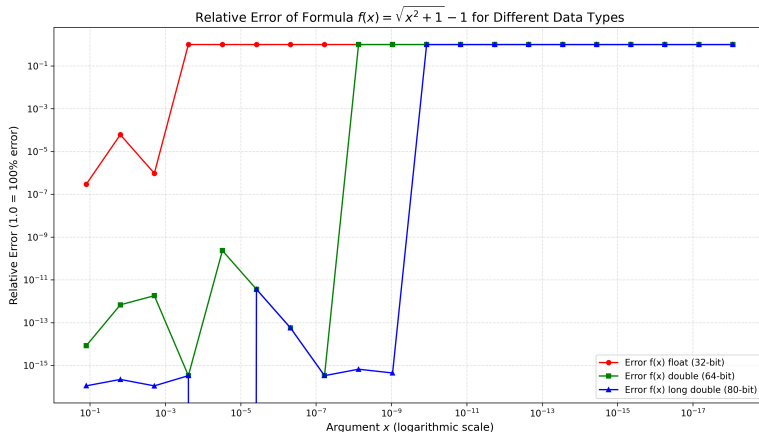
- Dla dużych x , operacja x^2 może spowodować **Overflow** ($+\infty$).
- W funkcji $g(x)$ prowadzi to do symbolu nieoznaczonego $\frac{\infty}{\infty}$, interpretowanego jako NaN.
- **Optymalizacja:** Dla ekstremalnie dużych x należy stosować przybliżenie asymptotyczne $f(x) \approx x - 1$.

Wyniki: Zanik wartości funkcji $f(x)$



Rysunek: Porównanie wartości obliczonych: $f(x)$ vs $g(x)$. Widoczny nagły spadek $f(x)$ do zera.

Wyniki: Błąd względny



Rysunek: Błąd względny funkcji $f(x)$ narastający wraz ze spadkiem wartości argumentu.

Analiza wpływu precyzji na stabilność

Z wykresów wynika wyraźna korelacja między długością mantysy a momentem wystąpienia błędu:

- **float (23 bity mantysy):** Całkowita utrata precyzji dla $x \approx 10^{-4}$.
- **double (52 bity mantysy):** Stabilność zachowana do $x \approx 10^{-8}$.
- **long double (64/80 bitów):** Najwyższa odporność na błędy zaokrągleń.

Wniosek: Zmiana typu na wyższy jedynie odsuwa problem w czasie, nie rozwiązując niestabilności samego algorytmu.

Zachowanie dla dużych argumentów (Overflow)

Wyniki eksperymentalne potwierdzają granice typów:

x	f_float	g_float	f_double	g_double
10^{10}	10^{10}	10^{10}	10^{10}	10^{10}
10^{30}	inf	NaN	10^{30}	10^{30}
10^{170}	inf	NaN	inf	NaN

- **inf** pojawia się w momencie przekroczenia zakresu przez x^2 .
- **NaN** w $g(x)$ wynika z dzielenia nieskończoności przez nieskończoność.

Szczegóły implementacyjne i biblioteczne

Ważnym aspektem jest dobór funkcji z biblioteki `<cmath>`:

- `std::sqrt(float)` - przeciążenie dla `float`, zapobiega niejawniej konwersji do `double`.
- Literały: `1.0f` (`float`), `1.0L` (`long double`) - kluczowe dla uniknięcia błędów precyzji już na etapie definicji stałych.
- Funkcje takie jak `std::hypot(x, 1.0)` w standardzie C++11 są dedykowane do stabilnego obliczania $\sqrt{x^2 + y^2}$ bez ryzyka przedwczesnego overflow.

- ❶ Tożsamość matematyczna nie jest równoznaczna z tożsamością numeryczną.
- ❷ **Unikanie odejmowania bliskich liczb:** Jest to najskuteczniejsza metoda eliminacji katastrofального obciążenia.
- ❸ **Analiza przedziałów:** W profesjonalnych bibliotekach (np. GSL, Boost) implementuje się funkcje hybrydowe, zmieniające wzór obliczeniowy w zależności od rzędu wielkości argumentu.
- ❹ Dobór typu danych powinien być świadomy i poparty analizą błędów, a nie tylko chęcią "zwiększenia dokładności".

Dziękuję za uwagę.